Namma Route

A. Project details

- Page slug: namma-route
- Category: Software Development
- Level: Moderate
- Time to build: 6 hrs
- Card summary (max 100 characters): Find the fastest route across Tamil Nadu and watch Dijkstra, A* and BFS search the map live.
- Page subtitle (one sentence, max 160 characters): Build a road and train route finder for all 38 districts of Tamil Nadu, then watch A* find the same fastest route as Dijkstra while checking far fewer cities.
- Related program: AI-Powered Software Engineering & DSA Program

B. Guide content

1. Objective (2 to 3 sentences)

Build Namma Route, a web app that finds the fastest road and train route between any two of 55 Tamil Nadu cities. You will write Dijkstra, A* and BFS by hand, then animate them on a map so people can watch the computer search. By the end, you can show that A* finds the same route as Dijkstra while checking far fewer cities.

2. Prerequisites (3 to 6 items)

- JavaScript basics: variables, functions, arrays, objects and loops
- HTML and CSS basics: building a simple page and styling it
- What a graph is: points (nodes) joined by lines (edges)
- What a queue is, and a rough idea of Big-O
- A laptop with Chrome or Edge, and free GitHub and Vercel accounts

3. Tools you will use (3 to 8 items)

- A code editor: VS Code or Antigravity
- Google Chrome and its DevTools console
- Python 3 or Node.js, to run a local server
- tn-data.js, the ready-made Tamil Nadu data file (55 cities, 86 road links, 60 rail links)

- Git and GitHub, to save and share your code
- Vercel, to put the app on a free live link

4. Steps (5 to 10)

Step 1: Set up the project and load the data

- Instruction: Create a folder named namma-route with index.html, styles.css and a js folder. Download tn-data.js from this page and save it as js/tn-data.js. It holds 55 cities (38 district headquarters and 17 towns), 86 road links, 60 rail links, map positions and the state outline. Start a local server with `python -m http.server 8000` and open http://localhost:8000. You need the server because browsers block JavaScript modules opened straight from a file.
- Code (optional):

```
<script type="module">
  import { CITIES, ROAD_LINKS, RAIL_LINKS } from './js/tn-data.js';
  console.log(CITIES.length, ROAD_LINKS.length, RAIL_LINKS.length);
</script>
```

- Checkpoint: The console prints 55 86 60 with no errors.

Step 2: Turn the cities into a graph

- Instruction: In js/graph.js, give every city a road node ("madurai:road") and every city with a station a rail node ("madurai:rail"). Join road nodes with road links and rail nodes with rail links, using each link's minutes as the weight. In Any mode, also join each city's road and rail nodes with a 30 minute switch edge. Then work out k, the smallest minutes per straight-line km across all 146 links. A* uses it in Step 5.
- Code (optional):

```
import { CITIES, ROAD_LINKS, RAIL_LINKS, TRANSFER_MINUTES } from './tn-
data.js';

export const byId = Object.fromEntries(CITIES.map(c => [c.id, c]));

// Straight-line km between two cities on a round Earth (haversine)
export function haversineKm(a, b) {
  const rad = d => d * Math.PI / 180;
  const dLat = rad(b.lat - a.lat), dLng = rad(b.lng - a.lng);
  const h = Math.sin(dLat / 2) ** 2 +
    Math.cos(rad(a.lat)) * Math.cos(rad(b.lat)) * Math.sin(dLng / 2) ** 2;
  return 2 * 6371 * Math.asin(Math.sqrt(h));
}
```

```
// k = the smallest (minutes / straight-line km) across every link
export const k = Math.min(...[...ROAD_LINKS, ...RAIL_LINKS]
  .map(l => l.min / haversineKm(byId[l.a], byId[l.b])));

// mode is 'any', 'road' or 'rail'
export function buildGraph(mode) {
  const adj = new Map();
  const link = (u, v, w) => {            // a two-way edge
    adj.get(u).push({ node: v, w });
    adj.get(v).push({ node: u, w });
  };
  for (const c of CITIES) {
    if (mode !== 'rail') adj.set(`${c.id}:road`, []);
    if (mode !== 'road' && c.rail) adj.set(`${c.id}:rail`, []);
  }
  if (mode !== 'rail') ROAD_LINKS.forEach(l => link(`${l.a}:road`,
`${l.b}:road`, l.min));
  if (mode !== 'road') RAIL_LINKS.forEach(l => link(`${l.a}:rail`,
`${l.b}:rail`, l.min));
  if (mode === 'any') CITIES.filter(c => c.rail)
    .forEach(c => link(`${c.id}:road`, `${c.id}:rail`, TRANSFER_MINUTES));
  for (const list of adj.values()) list.sort((x, y) => (x.node < y.node ? -1
: 1));
  return adj;
}

// Check: count nodes and one-way edges
// const g = buildGraph('any');
// console.log(k.toFixed(6), g.size, [...g.values()].reduce((s, l) => s +
l.length, 0));
```

- Checkpoint: k prints as 1.006192. In Any mode the graph has 104 nodes and 390 one-way edges. Road mode has 172 edges and Train mode has 120.

Step 3: Write a min-heap priority queue

- Instruction: Dijkstra and A* always need the waiting node with the lowest time. A min-heap gives it in O(log n) steps instead of scanning the whole list. Store items in an array: the parent of position i is at (i - 1) / 2 rounded down, and its children are at 2i + 1 and 2i + 2. push adds at the end and swaps upward while the item beats its parent. pop takes the top, moves the last item to the top and swaps it down with its smaller child. Break ties by node name so the same trip always gives the same route.
- Code (optional):

```
export class MinHeap {
  constructor() { this.a = []; }
  get size() { return this.a.length; }
  less(i, j) {                    // lower time wins; a tie goes to the
smaller name
    const x = this.a[i], y = this.a[j];
    return x.p !== y.p ? x.p < y.p : x.id < y.id;
  }
  swap(i, j) { [this.a[i], this.a[j]] = [this.a[j], this.a[i]]; }
  push(item) {                    // add at the end, then bubble up
    this.a.push(item);
    let i = this.a.length - 1;
    while (i > 0) {
      const parent = Math.floor((i - 1) / 2);
      if (!this.less(i, parent)) break;
      this.swap(i, parent);
      i = parent;
    }
  }
  // pop(): write it yourself. Take a[0], move the last item to the top,
  // then swap it down with its smaller child until neither child is smaller.
}
```

- Checkpoint: Push items with priorities 5, 1 and 3. Three pops return 1, then 3, then 5.

Step 4: Find the fastest route with Dijkstra

- Instruction: In js/algorithms.js, write Dijkstra as a generator function (function*). Put the start city's nodes in the heap at 0 minutes. Each loop, pop the lowest-time node, skip it if it is already settled, and stop when its city is the destination. For each neighbour, if the new time beats the best known time, save it, record where you came from in prev, and push it. The yield lines let Step 9 animate the search one step at a time. To read the final result, call next() until done is true. A for...of loop drops the return value.
- Code (optional):

```
import { MinHeap } from './heap.js';

export const cityOf = node => node.split(':')[0];   // 'madurai:rail' ->
'madurai'

export function* dijkstra(adj, from, target) {
  const dist = new Map(), prev = new Map(), done = new Set();
  const heap = new MinHeap();
  for (const n of [`${from}:road`, `${from}:rail`]) {
```

```
    if (adj.has(n)) { dist.set(n, 0); heap.push({ id: n, p: 0 }); }
  }
  while (heap.size) {
    const { id } = heap.pop();
    if (done.has(id)) continue;              // already settled
    done.add(id);
    yield { type: 'visit', id, minutes: dist.get(id) };
    if (cityOf(id) === target) return { end: id, minutes: dist.get(id), prev
};
    for (const { node, w } of adj.get(id)) {
      const t = dist.get(id) + w;
      if (t < (dist.get(node) ?? Infinity)) {
        dist.set(node, t);
        prev.set(node, id);
        heap.push({ id: node, p: t });
        yield { type: 'queue', id: node };
      }
    }
  }
  return null;                               // no route in this mode
}
```

- Checkpoint: Chennai to Madurai in Any mode returns 492 minutes. Chennai to Ooty returns 707 minutes in Any mode and 773 in Road mode.

Step 5: Add a sense of direction with A*

- Instruction: Copy dijkstra into a new function named astar and change only the heap priority. Use the time so far plus an estimate of the time left: k times the straight-line km from the node's city to the destination. k comes from the fastest link, so the estimate never overestimates and A* still finds the true fastest route. Count the unique cities each algorithm visits so you can compare them.
- Code (optional):

```
import { byId, haversineKm, k } from './graph.js';

// Inside astar: estimated minutes left from a node to the target
const h = node => k * haversineKm(byId[cityOf(node)], byId[target]);

// Then change the two heap pushes:
heap.push({ id: n, p: h(n) });               // start nodes (was p: 0)
heap.push({ id: node, p: t + h(node) });     // neighbours (was p: t)
```

- Checkpoint: Kanyakumari to Chennai in Any mode takes 738 minutes with both algorithms. Dijkstra checks 49 cities and A* checks 16.

Step 6: Find the fewest stops with BFS

- Instruction: BFS ignores time and counts stops. Build a neighbour list for each city from the links allowed in the current mode, sorted by name. Use a plain first-in, first-out queue instead of the heap, and mark each city the first time you reach it. Write it as a generator too, so it can be animated.
- Code (optional):

```
export function* bfs(neighbours, from, target) {   // neighbours: Map of city
-> sorted city list
  const prev = new Map([[from, null]]);
  const queue = [from];
  let head = 0;                                    // read pointer: first in,
first out
  while (head < queue.length) {
    const city = queue[head++];
    yield { type: 'visit', id: city };
    if (city === target) return { end: city, prev };
    for (const next of neighbours.get(city)) {
      if (!prev.has(next)) { prev.set(next, city); queue.push(next); }
    }
  }
  return null;
}
```

- Checkpoint: Chennai to Kanyakumari takes 10 stops by road and 11 by train. Coimbatore to Ooty is 3 stops both ways, even though the road takes 179 minutes and the hill train takes 329.

Step 7: Draw the Tamil Nadu map

- Instruction: Create an SVG with viewBox "0 0 1000 1330". Draw in layers from the bottom up: the state outline (STATE_OUTLINE), road lines, rail lines, then city dots and names. Use each city's x and y for drawing only, never for distance. Shift each line sideways by its offset so road and rail between the same two cities sit side by side. Zoom by changing the viewBox between 1x and 4x, and pan by dragging. To pick a city, turn the tap point into map units and choose the nearest city within 22 screen pixels.
- Code (optional):

```
// Shift a link sideways by its offset so road and rail sit side by side
const a = byId[link.a], b = byId[link.b];
const dx = b.x - a.x, dy = b.y - a.y, len = Math.hypot(dx, dy);
const nx = (-dy / len) * link.offset, ny = (dx / len) * link.offset;
// draw the line from (a.x + nx, a.y + ny) to (b.x + nx, b.y + ny)

// Tap to pick: screen pixels to map units, correct at any zoom or pan
svg.addEventListener('click', e => {
  const ctm = svg.getScreenCTM();
  const p = new DOMPoint(e.clientX,
e.clientY).matrixTransform(ctm.inverse());
  let best = null, bestD = 22 / ctm.a;      // 22 screen pixels, in map units
  for (const c of CITIES) {
    const d = Math.hypot(c.x - p.x, c.y - p.y);
    if (d < bestD) { best = c; bestD = d; }
  }
  if (best) pickCity(best.id);
});
```

- Checkpoint: The map shows 55 city dots, 86 road lines and 60 rail lines. Tapping beside Madurai picks Madurai at 1x zoom and at 3x zoom.

Step 8: Build the trip planner and result card

- Instruction: Add Any, Road and Train tabs, From and To boxes with search, and tabs for Dijkstra, A* and BFS. Make the search match each city's aliases too, so "Trichy" finds Tiruchirappalli. In Train mode, dim the 6 cities with no station and stop them being picked. After a search, walk back through prev from the destination to rebuild the route. Join back-to-back hops of the same type into one step, and add a switch step wherever the type changes. Show the total time, km, stops and switches.
- Code (optional):

```
// Rebuild the route by walking back from the destination node
const path = [];
for (let n = result.end; n; n = result.prev.get(n)) path.unshift(n);
// Chennai to Ooty (Any) gives:
// chennai:rail > ... > mettupalayam:rail > mettupalayam:road > coonoor:road
> ooty:road
```

- Checkpoint: Chennai to Ooty in Any mode shows about 11 h 47 min and 587 km with 1 switch: a train to Mettupalayam, a switch to road, then road to Ooty via Coonoor.

Step 9: Animate the search and add Compare mode

- Instruction: Call next() on the generator every 45 ms (Slow 140 ms, Fast 12 ms). On a visit event, pulse that city, update the cities-checked counter and show one line of narration. On a queue event, draw a faint ring. Pause clears the timer, Play restarts it, and Step calls next() once. When the search ends, draw the route line along the path. For Compare, run Dijkstra, pause 600 ms, clear the map, run A*, then show both counts side by side.
- Code (optional):

```
let gen, timer, speedMs = 45;              // Slow 140, Normal 45, Fast 12

function run(algorithm, ...args) {         // dijkstra, astar or bfs
  clearTimeout(timer);
  gen = algorithm(...args);
  tick();
}

function tick() {
  const { value, done } = gen.next();
  if (done) return showResult(value);      // the route, or null if none
  if (value.type === 'visit') pulseCity(value.id);   // plus counter and
narration
  if (value.type === 'queue') ringCity(value.id);
  timer = setTimeout(tick, speedMs);
}
```

- Checkpoint: Compare on Kanyakumari to Chennai ends with "Dijkstra checked 49 cities. A* checked 16. Same route, 67% less work."

Step 10: Add themes and publish your app

- Instruction: Put every colour in CSS variables, with a light set on :root and a dark set on [data-theme="dark"]. Add a toggle button and save the choice in localStorage. Add this footer disclaimer: "Simplified map for learning. Not official. Distances and times are approximate and do not include waiting for trains or buses." Push the folder to a new GitHub repository. In Vercel, click Add New, then Project, import the repository and click Deploy.
- Code (optional):

```
:root { --bg: #FFFFFF; --text: #111827; --brand: #059669; }
[data-theme="dark"] { --bg: #0F172A; --text: #F1F5F9; --brand: #10B981; }
body { background: var(--bg); color: var(--text); }
```

```
function toggleTheme() {
  const root = document.documentElement;
  const next = root.dataset.theme === 'dark' ? 'light' : 'dark';
  root.dataset.theme = next;
  try { localStorage.setItem('nr-theme', next); } catch (e) { /* storage
blocked */ }
}
```

- Checkpoint: Your live link opens on a phone. The theme you pick stays after a refresh, and the disclaimer is always visible.

5. Expected result

- Demo video link: [To add after the video is uploaded]
- Your app should:
  - Show a Tamil Nadu map with 55 cities, road lines and train lines
  - Let people pick From and To by typing a name or alias, or by tapping the map
  - Offer Any, Road and Train modes, with the 6 cities that have no station dimmed in Train mode
  - Find routes with Dijkstra, A* and BFS, and show the time, km, stops, switches and steps in plain words
  - Animate the search live, with Play, Pause, Step and three speeds
  - Compare Dijkstra and A* on one trip: for Kanyakumari to Chennai, 49 cities checked against 16, with the same 738 minute route
  - Work on phone and laptop, in light and dark themes, with the disclaimer always visible
  - Open from a public live link

6. LinkedIn post text

I just built Namma Route, a road and train route finder for Tamil Nadu, using the free project guide from @Wrench Wise.

It finds the fastest way between 55 cities and lets you watch Dijkstra, A* and BFS search the map step by step.

Try it: [your live link]

#WrenchWiseProjects #DSA #JavaScript